

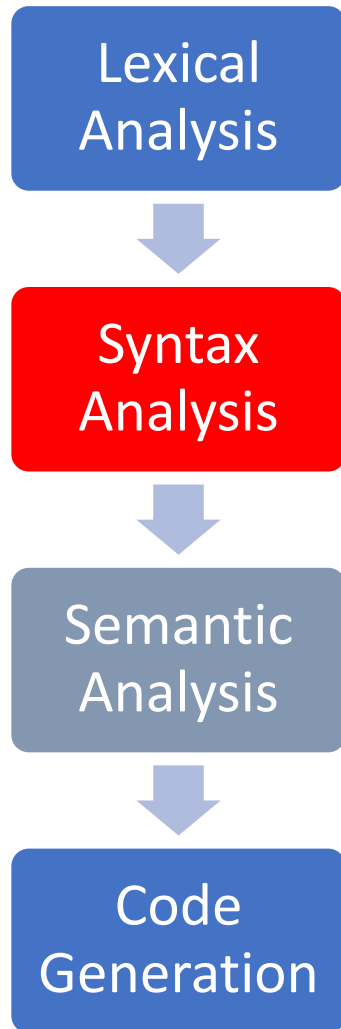
Compiler construction experimentation

Lecture 2. Syntax analysis

ONE LOVE. ONE FUTURE.

- Overview
- KPL's grammar
- Parser implementation

- **Compilation process – syntax analysis**

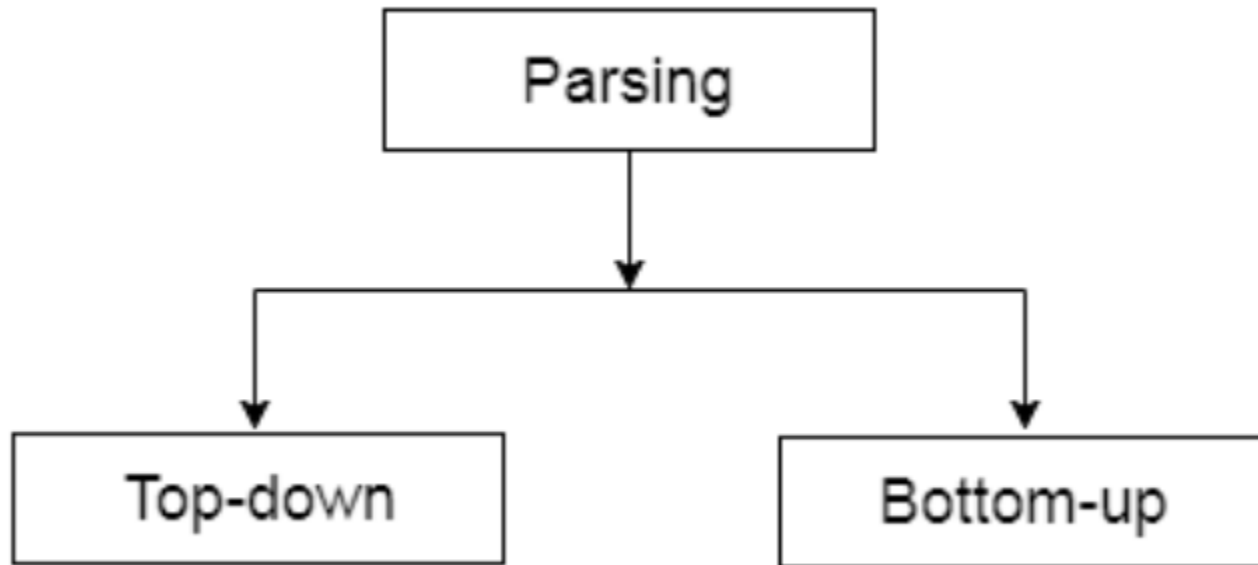


Ensure that the given program is syntactical correct

Syntactical structure of a PL is normally represented as a grammar

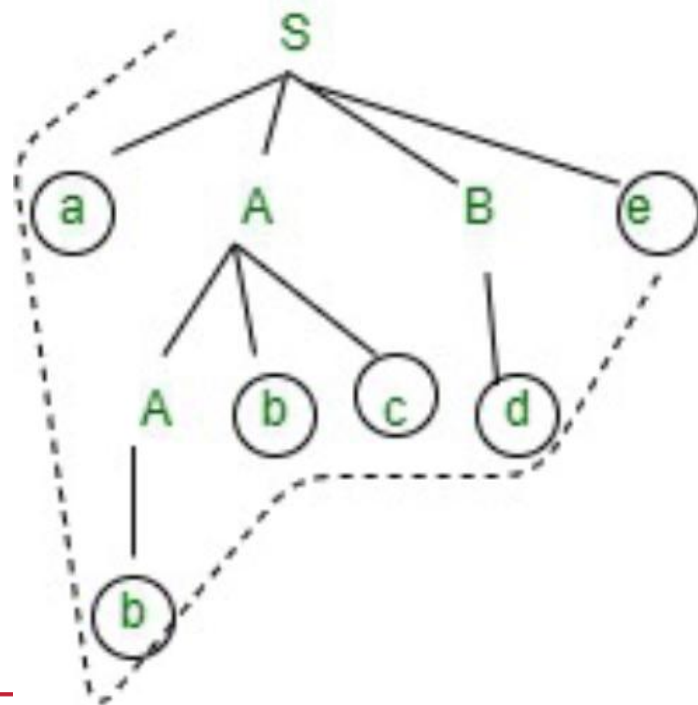
Serving next phases

Types of parsing techniques

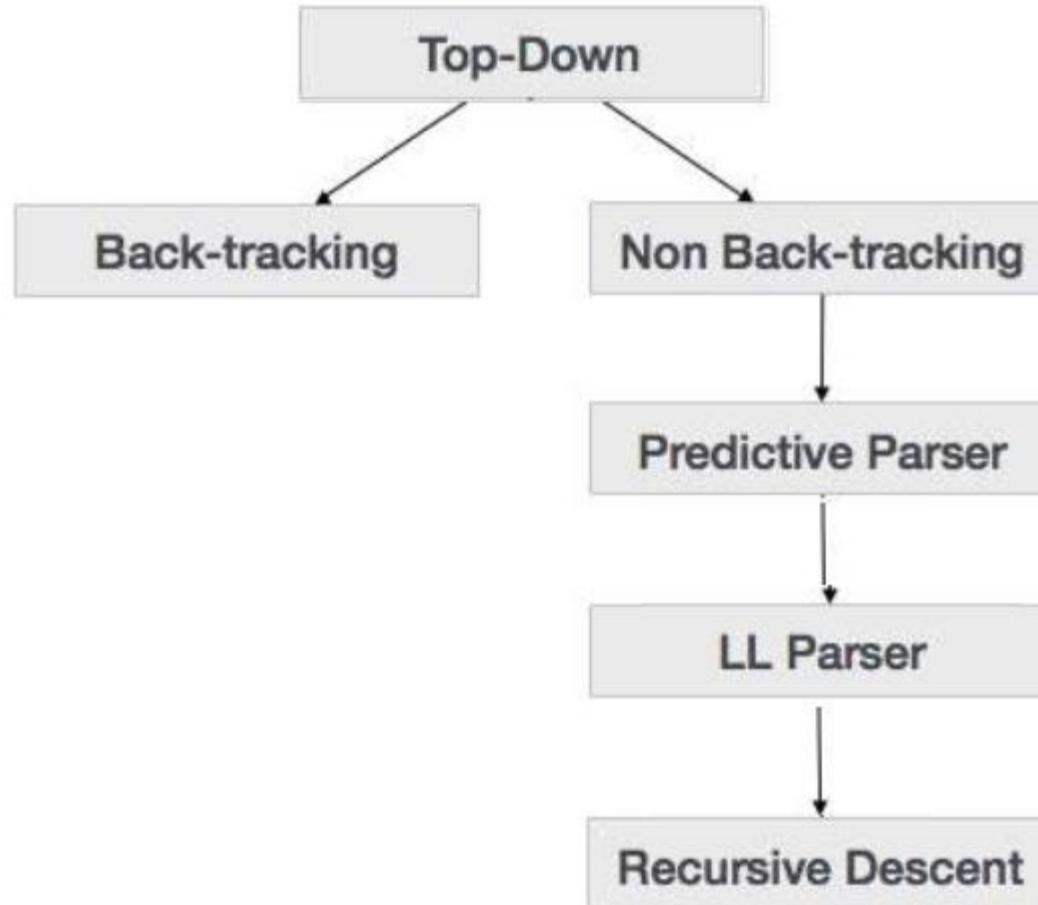


Top-down parsing

- The syntax tree is constructed from root to leaves, the source program is read from left to right
- Syntax tree construction is based on left-most derivation
 - Question: when expanding a non-terminal, if we have more than one rule to take, which one will we choose?
- Ex: Grammar G
 - G: (1) $S \rightarrow a A B e$
(2, 3) $A \rightarrow A b c | b$
(4) $B \rightarrow d$
 - Input: abbcde



Top-down parsing techniques



- A top-down parser
- Implemented as a set of procedures, each procedure corresponds to a non-terminal
 - For a single-rule terminal, the corresponding procedure is just a sequence of symbol reading/analysing steps
 - For a multiple-rule terminal, the procedure will have to choose a correct rule to follow using look ahead token (terminal symbol from source program)
 - Parsing process begins with the calling of start-symbol procedure

- Production

$\langle \text{phrase1} \rangle \rightarrow E$

The parsing procedure would be

```
void compilePhrase1( )  
{    /* reading/analyzing E */    }
```

- E is a sequence of non-terminals and terminals
 - For a terminal, we need to check the look ahead symbol from input
 - For a non-terminal, we simply call its parsing procedure
 - Left recursion is prohibited in the grammar for recursive descent technique

Analysing a sequence of symbols

- Suppose that we have to analyze a sequence: $Y_1 Y_2 Y_3 \dots Y_n$
- Each non-terminal Y has a parsing procedure
`compileY`
- Each terminal Y can be read by
`eat(Y)`
 - `if (lookAhead == y)`
 - `then getNextToken()`
 - `else SyntaxError()`

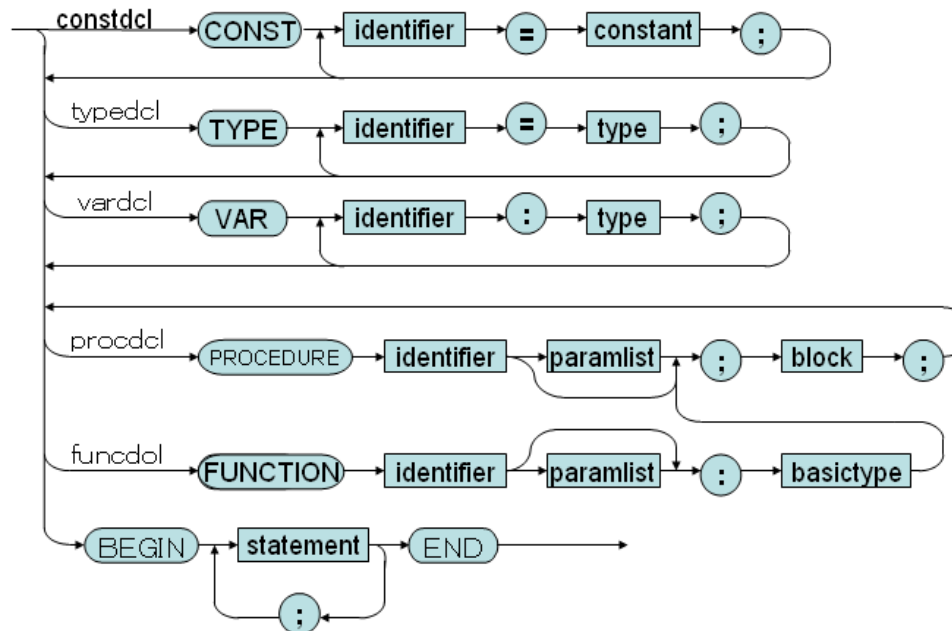


Syntax diagram of KPL

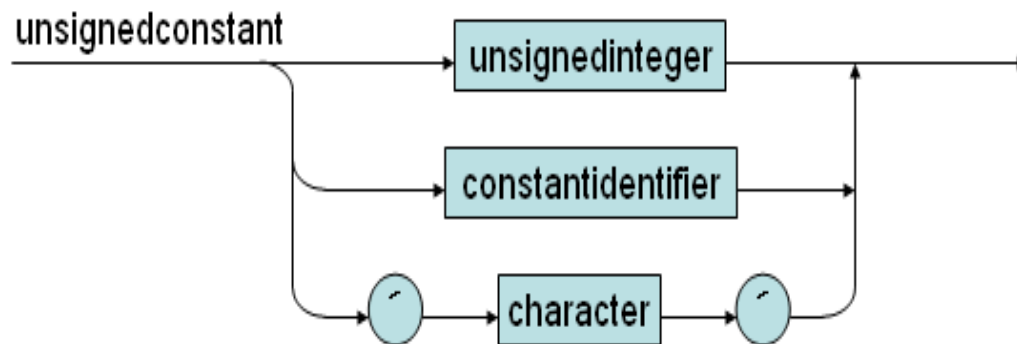
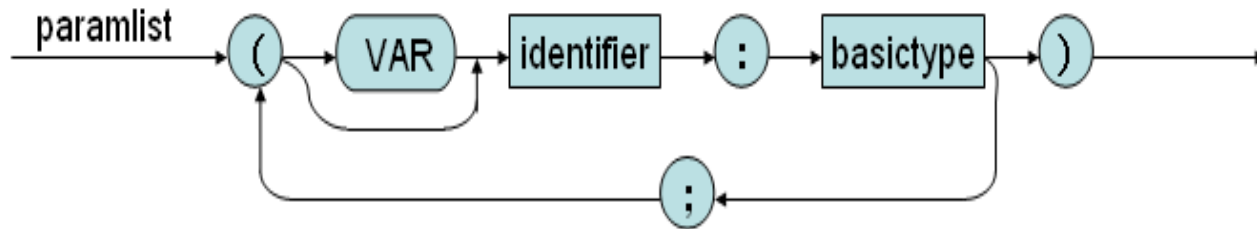
program



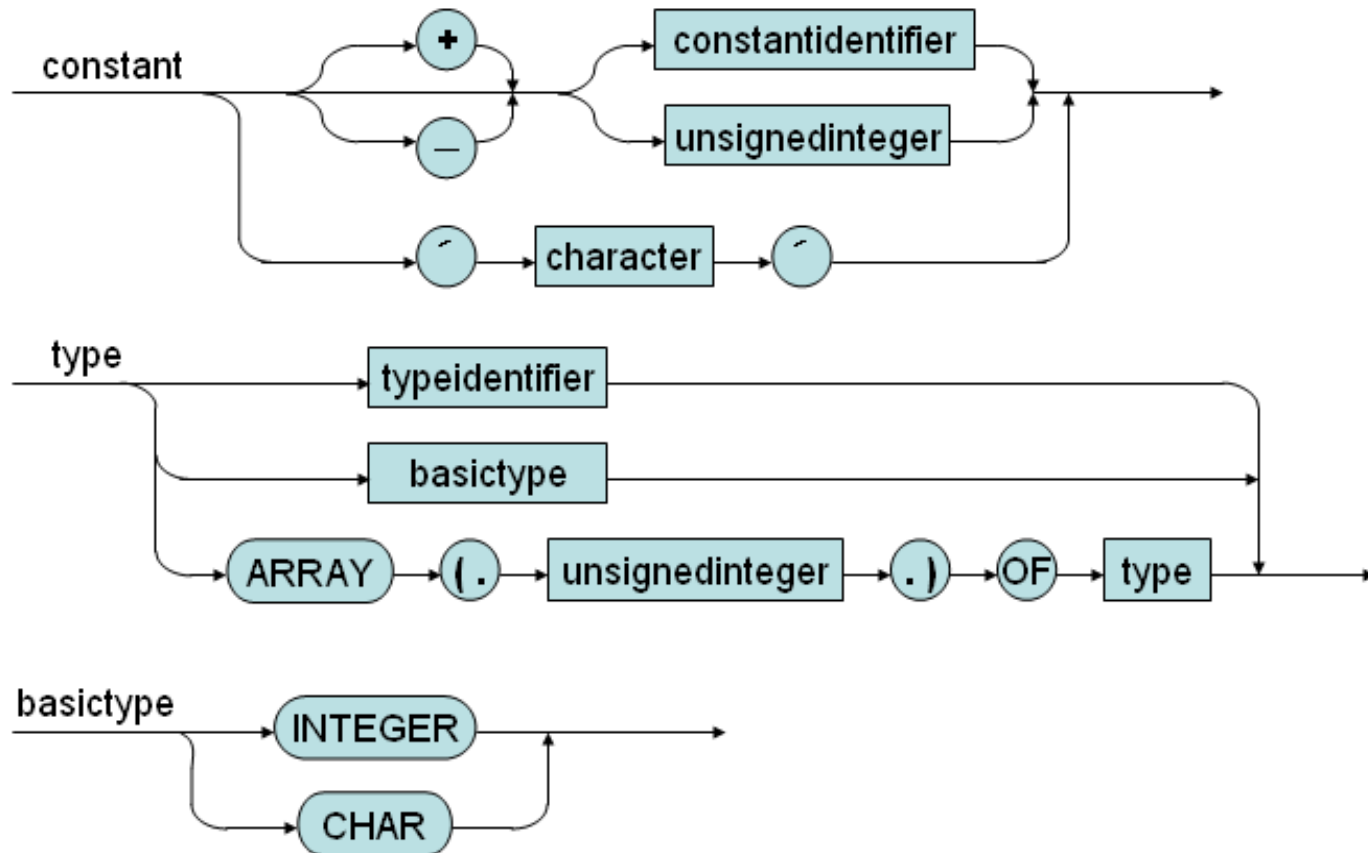
block



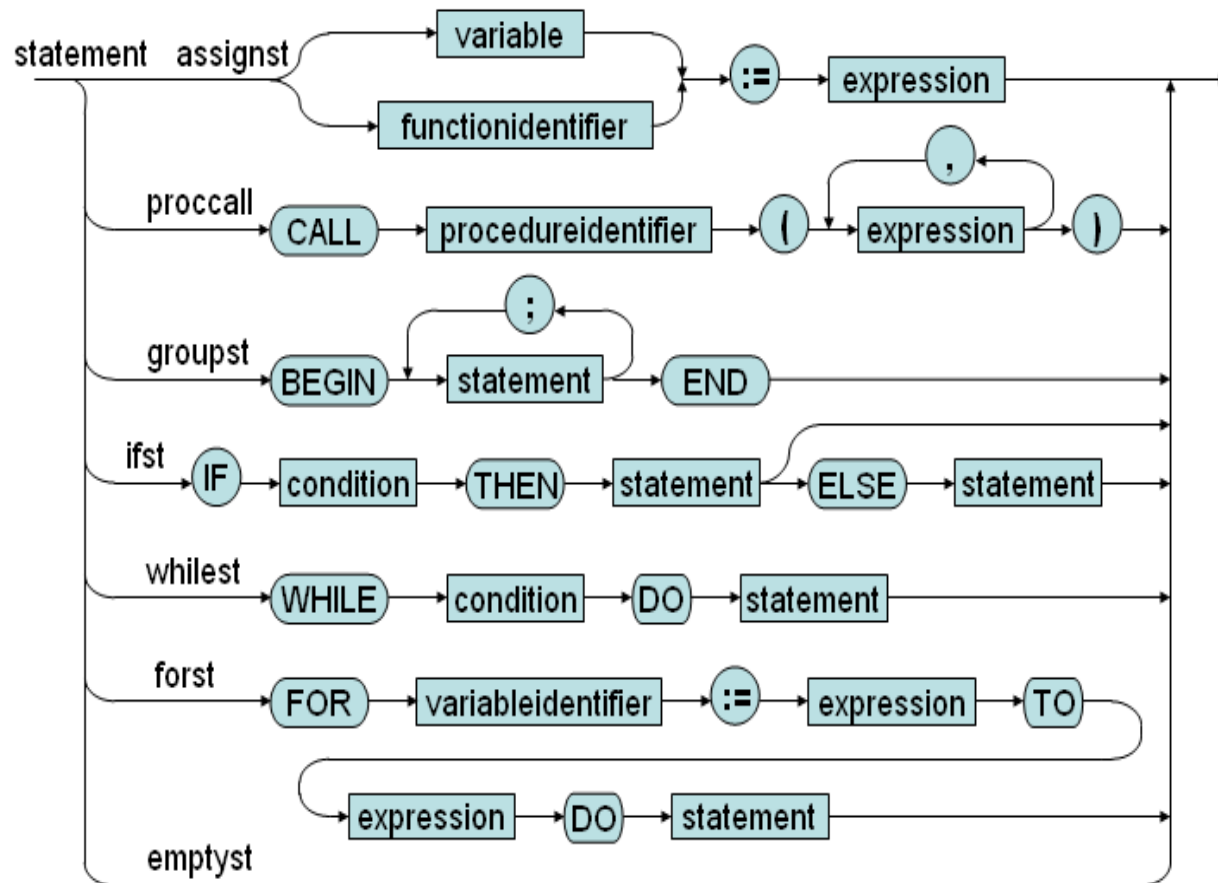
Syntax diagram of KPL



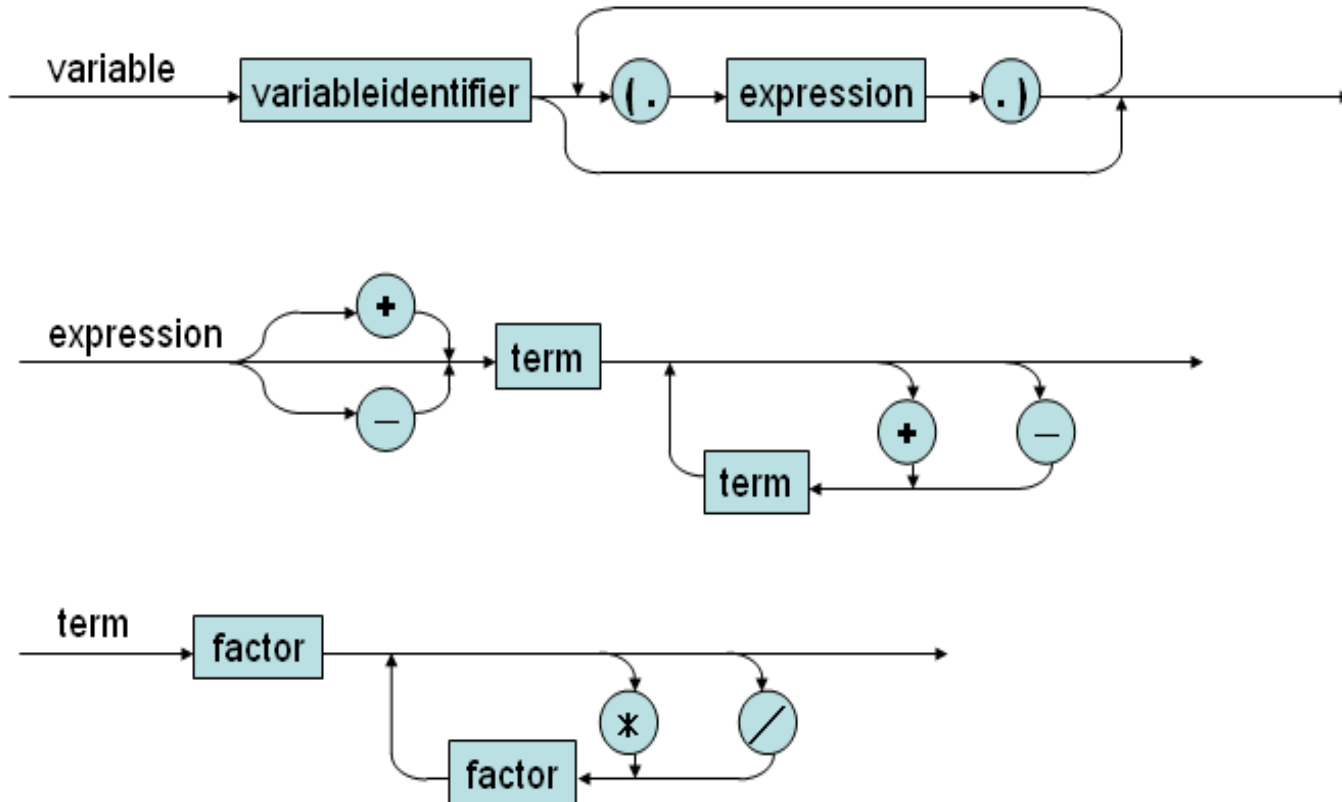
Syntax diagram of KPL



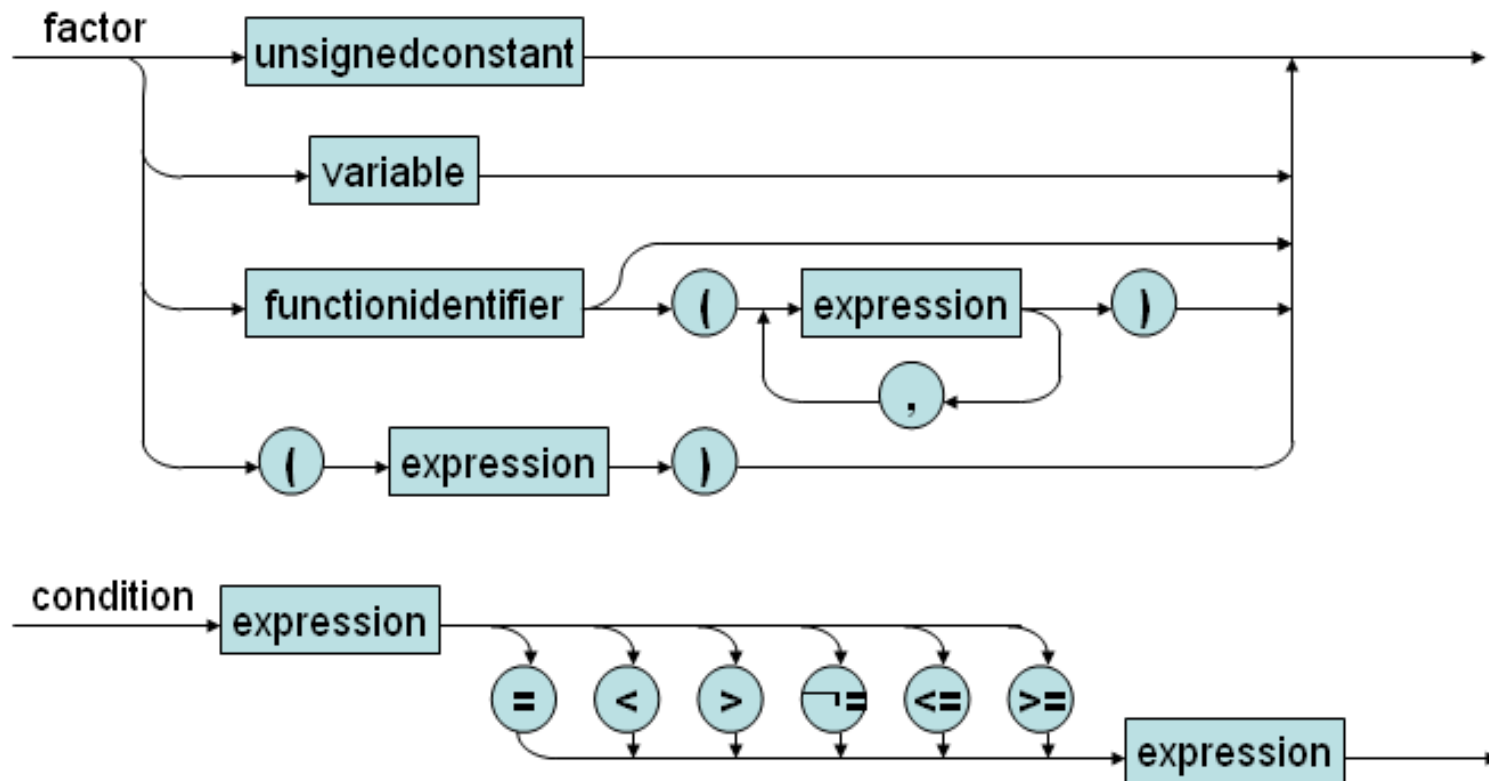
Syntax diagram of KPL



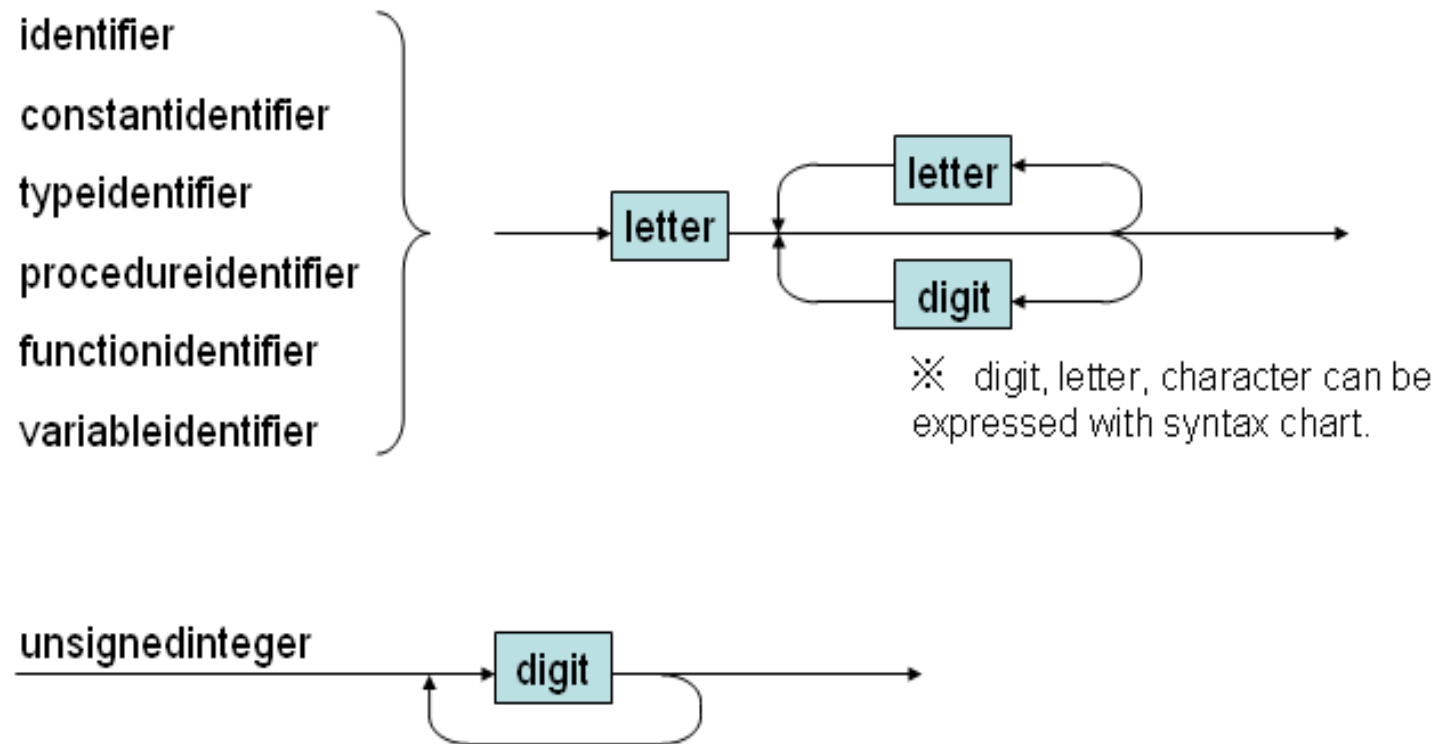
Syntax diagram of KPL



Syntax diagram of KPL



Syntax diagram of KPL



- No left recursion production
 - Left recursion production: “ $A \rightarrow Aw$ ”
- Left refactoring
 - Before: $A \rightarrow Bu \mid Bv$
 - After: $A \rightarrow BA'$; $A' \rightarrow u \mid v$

- 01) $\langle \text{Prog} \rangle ::= \text{KW_PROGRAM TK_IDENT SB_SEMICOLON} \langle \text{Block} \rangle \text{SB_PERIOD}$
- 02) $\langle \text{Block} \rangle ::= \text{KW_CONST} \langle \text{ConstDecl} \rangle \langle \text{ConstDecls} \rangle \langle \text{Block2} \rangle$
- 03) $\langle \text{Block} \rangle ::= \langle \text{Block2} \rangle$
- 04) $\langle \text{Block2} \rangle ::= \text{KW_TYPE} \langle \text{TypeDecl} \rangle \langle \text{TypeDecls} \rangle \langle \text{Block3} \rangle$
- 05) $\langle \text{Block2} \rangle ::= \langle \text{Block3} \rangle$
- 06) $\langle \text{Block3} \rangle ::= \text{KW_VAR} \langle \text{VarDecl} \rangle \langle \text{VarDecls} \rangle \langle \text{Block4} \rangle$
- 07) $\langle \text{Block3} \rangle ::= \langle \text{Block4} \rangle$
- 08) $\langle \text{Block4} \rangle ::= \langle \text{SubDecls} \rangle \langle \text{Block5} \rangle$
- 09) $\langle \text{Block4} \rangle ::= \langle \text{Block5} \rangle$
- 10) $\langle \text{Block5} \rangle ::= \text{KW_BEGIN} \langle \text{Statements} \rangle \text{KW_END}$

- 11) $\langle \text{ConstDecls} \rangle ::= \langle \text{ConstDecl} \rangle \langle \text{ConstDecls} \rangle$
- 12) $\langle \text{ConstDecls} \rangle ::= \varepsilon$
- 13) $\langle \text{ConstDecl} \rangle ::= \text{TK_IDENT SB_EQUAL} \langle \text{Constant} \rangle \text{SB_SEMICOLON}$
- 14) $\langle \text{TypeDecls} \rangle ::= \langle \text{TypeDecl} \rangle \langle \text{TypeDecls} \rangle$
- 15) $\langle \text{TypeDecls} \rangle ::= \varepsilon$
- 16) $\langle \text{TypeDecl} \rangle ::= \text{TK_IDENT SB_EQUAL} \langle \text{Type} \rangle \text{SB_SEMICOLON}$
- 17) $\langle \text{VarDecls} \rangle ::= \langle \text{VarDecl} \rangle \langle \text{VarDecls} \rangle$
- 18) $\langle \text{VarDecls} \rangle ::= \varepsilon$
- 19) $\langle \text{VarDecl} \rangle ::= \text{TK_IDENT SB_COLON} \langle \text{Type} \rangle \text{SB_SEMICOLON}$
- 20) $\langle \text{SubDecls} \rangle ::= \langle \text{FunDecl} \rangle \langle \text{SubDecls} \rangle$
- 21) $\langle \text{SubDecls} \rangle ::= \langle \text{ProcDecl} \rangle \langle \text{SubDecls} \rangle$
- 22) $\langle \text{SubDecls} \rangle ::= \varepsilon$

```
23) <FunDecl> ::= KW_FUNCTION TK_IDENT <Params> SB_COLON
               <BasicType> SB_SEMICOLON <Block> SB_SEMICOLON
```

24) **<ProcDecl> ::= KW_PROCEDURE TK_IDENT <Params> SB_SEMICOLON
<Block> SB_SEMICOLON**

25) $\langle \text{Params} \rangle ::= \text{SB_LPAR } \langle \text{Param} \rangle \langle \text{Params2} \rangle \text{SB_RPAR}$

26) $\langle \text{Params} \rangle ::= \varepsilon$

27) $\langle \text{Params2} \rangle ::= \text{SB SEMICOLON } \langle \text{Param} \rangle \langle \text{Params2} \rangle$

28) $\langle \text{Params2} \rangle ::= \varepsilon$

29) **<Param>** ::= TK IDENT SB COLON **<BasicType>**

30) **<Param>** ::= KW VAR TK IDENT SB COLON **<BasicType>**

Grammar of KPL in BNF

- 31) `<Type> ::= KW_INTEGER`
- 32) `<Type> ::= KW_CHAR`
- 33) `<Type> ::= TK_IDENT`
- 34) `<Type> ::= KW_ARRAY SB_LSEL TK_NUMBER SB_RSEL KW_OF <Type>`

- 35) `<BasicType> ::= KW_INTEGER`
- 36) `<BasicType> ::= KW_CHAR`

- 37) `<UnsignedConstant> ::= TK_NUMBER`
- 38) `<UnsignedConstant> ::= TK_IDENT`
- 39) `<UnsignedConstant> ::= TK_CHAR`

- 40) `<Constant> ::= SB_PLUS <Constant2>`
- 41) `<Constant> ::= SB_MINUS <Constant2>`
- 42) `<Constant> ::= <Constant2>`
- 43) `<Constant> ::= TK_CHAR`

- 44) `<Constant2> ::= TK_IDENT`
- 45) `<Constant2> ::= TK_NUMBER`

- 46) $\langle \text{Statements} \rangle ::= \langle \text{Statement} \rangle \langle \text{Statements2} \rangle$
- 47) $\langle \text{Statements2} \rangle ::= \text{SB_SEMICOLON} \langle \text{Statement} \rangle \langle \text{Statements2} \rangle$
- 48) $\langle \text{Statements2} \rangle ::= \varepsilon$
- 49) $\langle \text{Statement} \rangle ::= \langle \text{AssignSt} \rangle$
- 50) $\langle \text{Statement} \rangle ::= \langle \text{CallSt} \rangle$
- 51) $\langle \text{Statement} \rangle ::= \langle \text{GroupSt} \rangle$
- 52) $\langle \text{Statement} \rangle ::= \langle \text{IfSt} \rangle$
- 53) $\langle \text{Statement} \rangle ::= \langle \text{WhileSt} \rangle$
- 54) $\langle \text{Statement} \rangle ::= \langle \text{ForSt} \rangle$
- 55) $\langle \text{Statement} \rangle ::= \varepsilon$

- 56) `<AssignSt> ::= <Variable> SB_ASSIGN <Expression>`
- 57) `<AssignSt> ::= TK_IDENT SB_ASSIGN <Expression>`
- 58) `<CallSt> ::= KW_CALL TK_IDENT <Arguments>`
- 59) `<GroupSt> ::= KW_BEGIN <Statements> KW_END`
- 60) `<IfSt> ::= KW_IF <Condition> KW_THEN <Statement> <ElseSt>`
- 61) `<ElseSt> ::= KW_ELSE <Statement>`
- 62) `<ElseSt> ::=  $\epsilon$`
- 63) `<WhileSt> ::= KW_WHILE <Condition> KW_DO <Statement>`
- 64) `<ForSt> ::= KW_FOR TK_IDENT SB_ASSIGN <Expression> KW_TO
 <Expression> KW_DO <Statement>`

- 65) $\langle \text{Arguments} \rangle ::= \text{SB_LPAR } \langle \text{Expression} \rangle \langle \text{Arguments2} \rangle \text{SB_RPAR}$
- 66) $\langle \text{Arguments} \rangle ::= \varepsilon$
- 67) $\langle \text{Arguments2} \rangle ::= \text{SB_COMMA } \langle \text{Expression} \rangle \langle \text{Arguments2} \rangle$
- 68) $\langle \text{Arguments2} \rangle ::= \varepsilon$
- 69) $\langle \text{Condition} \rangle ::= \langle \text{Expression} \rangle \langle \text{Condition2} \rangle$
- 70) $\langle \text{Condition2} \rangle ::= \text{SB_EQ } \langle \text{Expression} \rangle$
- 71) $\langle \text{Condition2} \rangle ::= \text{SB_NEQ } \langle \text{Expression} \rangle$
- 72) $\langle \text{Condition2} \rangle ::= \text{SB_LE } \langle \text{Expression} \rangle$
- 73) $\langle \text{Condition2} \rangle ::= \text{SB_LT } \langle \text{Expression} \rangle$
- 74) $\langle \text{Condition2} \rangle ::= \text{SB_GE } \langle \text{Expression} \rangle$
- 75) $\langle \text{Condition2} \rangle ::= \text{SB_GT } \langle \text{Expression} \rangle$

- 76) $\langle \text{Expression} \rangle ::= \text{SB_PLUS } \langle \text{Expression2} \rangle$
- 77) $\langle \text{Expression} \rangle ::= \text{SB_MINUS } \langle \text{Expression2} \rangle$
- 78) $\langle \text{Expression} \rangle ::= \langle \text{Expression2} \rangle$

- 79) $\langle \text{Expression2} \rangle ::= \langle \text{Term} \rangle \langle \text{Expression3} \rangle$

- 80) $\langle \text{Expression3} \rangle ::= \text{SB_PLUS } \langle \text{Term} \rangle \langle \text{Expression3} \rangle$
- 81) $\langle \text{Expression3} \rangle ::= \text{SB_MINUS } \langle \text{Term} \rangle \langle \text{Expression3} \rangle$
- 82) $\langle \text{Expression3} \rangle ::= \varepsilon$

- 83) $\langle \text{Term} \rangle ::= \langle \text{Factor} \rangle \langle \text{Term2} \rangle$

- 84) $\langle \text{Term2} \rangle ::= \text{SB_TIMES } \langle \text{Factor} \rangle \langle \text{Term2} \rangle$
- 85) $\langle \text{Term2} \rangle ::= \text{SB_SLASH } \langle \text{Factor} \rangle \langle \text{Term2} \rangle$
- 86) $\langle \text{Term2} \rangle ::= \varepsilon$
- 87) $\langle \text{Factor} \rangle ::= \langle \text{UnsignedConstant} \rangle$
- 88) $\langle \text{Factor} \rangle ::= \langle \text{Variable} \rangle$
- 89) $\langle \text{Factor} \rangle ::= \langle \text{FunctionApptication} \rangle$
- 90) $\langle \text{Factor} \rangle ::= \text{SB_LPAR } \langle \text{Expression} \rangle \text{SB_RPAR}$

91) $\langle \text{Variable} \rangle ::= \text{TK_IDENT } \langle \text{Indexes} \rangle$

92) $\langle \text{FunctionApplication} \rangle ::= \text{TK_IDENT } \langle \text{Arguments} \rangle$

93) $\langle \text{Indexes} \rangle ::= \text{SB_LSEL } \langle \text{Expression} \rangle \text{ SB_RSEL } \langle \text{Indexes} \rangle$

94) $\langle \text{Indexes} \rangle ::= \varepsilon$

- KPL has some predefined functions/procedures for input/output

- Input:

- ReadI: Read and return an integer
- ReadC: Read and return a character

Ex.

```
var a: integer;  
a:= ReadI;
```

- Output:

- WriteI: Print an integer in the given parameter
- WriteC: Print a character in the given parameter
- WriteLn: Print a newline character.

Ex.

```
call WriteI(a);  
call WriteLn;
```

- Compute a square of a number.
- Total squares of the first N natural numbers

```
program example5;
var n : integer; i: integer; sum: integer;

function f(k : integer) : integer;
begin
    f := k * k;
end;

BEGIN
    n := readI;
    sum := 0;
    for i:=1 to n do
        sum:= sum + f(i);
    call writeln;
    call writeI(sum);
END. (* example*)
```

Implement a recursive descent parser for KPL

- Technically KPL is a LL(1)
- Implementing a recursive descent parser:
 - *lookAhead*
 - Eating terminals
 - Analyzing non-terminals
 - Analyzing table
 - Computing FIRST() và FOLLOW()

- Example

02) Block ::= KW_CONST ConstDecl ConstDecls Block2 =>RHS1

03) Block ::= Block2 =>RHS2

FIRST(RHS1)={KW_CONST}

FIRST(RHS2)={KW_TYPE, KW_VAR, KW_FUNCTION, KW_PROCEDURE, KW_BEGIN}

FIRST(RHS1) $\cap$ FIRST(RHS2) = $\emptyset$

LookAhead =KW_BEGIN =>RHS2 được chọn =>LL(1)

- Next token from input

```
Token *currentToken;  
Token *lookAhead;  
  
void scan(void) {  
    Token* tmp = currentToken;  
    currentToken = lookAhead;  
    lookAhead = getValidToken();  
    free(tmp);  
}
```



```
void eat(TokenType tokenType) {  
    if (lookAhead->tokenType == tokenType) {  
        printToken(lookAhead);  
        scan();  
    } else  
        missingToken(tokenType, lookAhead->lineNo, lookAhead->colNo);  
}
```

```
int compile(char *fileName) {  
    if (openInputStream(fileName) == IO_ERROR)  
        return IO_ERROR;  
  
    currentToken = NULL;  
    lookAhead = getValidToken();  
  
    compileProgram();  
  
    free(currentToken);  
    free(lookAhead);  
    closeInputStream();  
    return IO_SUCCESS;  
}
```

Program

1) $\langle \text{Prog} \rangle ::= \text{KW_PROGRAM TK_IDENT SB_SEMICOLON } \langle \text{Block} \rangle \text{ SB_PERIOD}$

```
void compileProgram(void) {  
    assert("Parsing a Program ....");  
    eat(KW_PROGRAM);  
    eat(TK_IDENT);  
    eat(SB_SEMICOLON);  
    compileBlock();  
    eat(SB_PERIOD);  
    assert("Program parsed!");  
}
```

Basic Type

35) `<BasicType> ::= KW_INTEGER`

36) `<BasicType> ::= KW_CHAR`

```
void compileBasicType(void) {  
    switch (lookAhead->tokenType) {  
        case KW_INTEGER:  
            eat(KW_INTEGER);  
            break;  
        case KW_CHAR:  
            eat(KW_CHAR);  
            break;  
        default:  
            error(ERR_INVALIDBASICTYPE, lookAhead->lineNo,  
lookAhead->colNo);  
            break;  
    }  
}
```

Analyzing statement

Statement: 7 production including empty production

FIRST(<Statement>) = {TK_IDENT, KW_CALL, KW_BEGIN, KW_IF, KW_WHILE,
KW_FOR, ϵ }

FOLLOW(<Statement>) = {SB_SEMICOLON, KW_END, KW_ELSE}

/* Predict parse table for Expression */

Input	Production

TK_IDENT	49) <Statement> ::= <AssignSt>
KW_CALL	50) <Statement> ::= <CallSt>
KW_BEGIN	51) <Statement> ::= <GroupSt>
KW_IF	52) <Statement> ::= <IfSt>
KW_WHILE	53) <Statement> ::= <Whilst?
KW_FOR	54) <Statement> ::= <ForSt?

SB_SEMICOLON	55) ϵ
KW_END	55) ϵ
KW_ELSE	55) ϵ

Khác	Error

Analyzing statement

```
void compileStatement(void)
{
    switch (lookAhead-
>tokenType) {
        case TK_IDENT:
            compileAssignSt();
            break;
        case KW_CALL:
            compileCallSt();
            break;
        case KW_BEGIN:
            compileGroupSt();
            break;
        case KW_IF:
            compileIfSt();
            break;
        case KW_WHILE:
            compileWhileSt();
            break;
```

```
        case KW_FOR:
            compileForSt();
            break;
            // check FOLLOW tokens
        case SB_SEMICOLON:
        case KW_END:
        case KW_ELSE:
            break;
            // Error occurs
        default:
            error(ERR_INVALIDSTATEMENT,
lookAhead->lineNo,
lookAhead->colNo);
            break;
    }
}
```

10) $\langle \text{ConstDecls} \rangle ::= \langle \text{ConstDecl} \rangle \langle \text{ConstDecls} \rangle$

11) $\langle \text{ConstDecls} \rangle ::= \varepsilon$

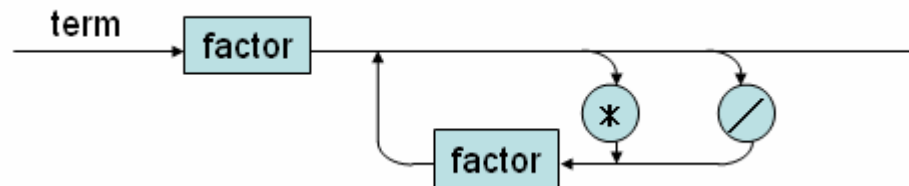
```
void compileConstDecls(void) {  
    while (lookAhead->tokenType == TK_IDENT)  
        compileConstDecl();  
}
```

83) $\langle \text{Term} \rangle ::= \langle \text{Factor} \rangle \langle \text{Term2} \rangle$

84) $\langle \text{Term2} \rangle ::= \text{SB_TIMES} \langle \text{Factor} \rangle \langle \text{Term2} \rangle$

85) $\langle \text{Term2} \rangle ::= \text{SB_SLASH} \langle \text{Factor} \rangle \langle \text{Term2} \rangle$

86) $\langle \text{Term2} \rangle ::= \varepsilon$



Combining

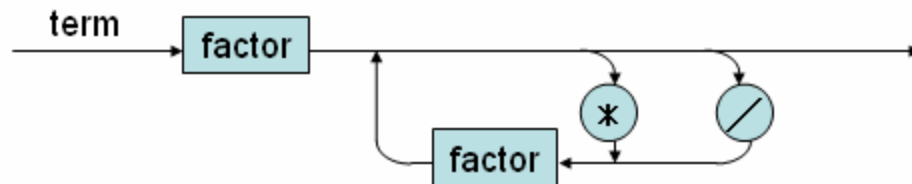
```
void compileTerm(void)
{ compileFactor();
  compileTerm2();
}

void compileTerm2(void) {
  switch (lookAhead->tokenType) {
    case SB_TIMES:
      eat(SB_TIMES);
      compileFactor();
      compileTerm2();
      break;
    case SB_SLASH:
      eat(SB_SLASH);
      compileFactor();
      compileTerm2();
      break;
    // check the FOLLOW set
    case SB_PLUS:
    case SB_MINUS:
    case KW_TO:
    case KW_DO:
      case SB_RPAR:
        case SB_COMMA:
        case SB_EQ:
        case SB_NEQ:
        case SB_LE:
        case SB_LT:
        case SB_GE:
        case SB_GT:
        case SB_RSEL:
        case SB_SEMICOLON:
        case KW_END:
        case KW_ELSE:
        case KW_THEN:
          break;
        default:
          error(ERR_INVALIDTERM, lookAhead->lineNo, lookAhead->colNo);
          }
  }
```

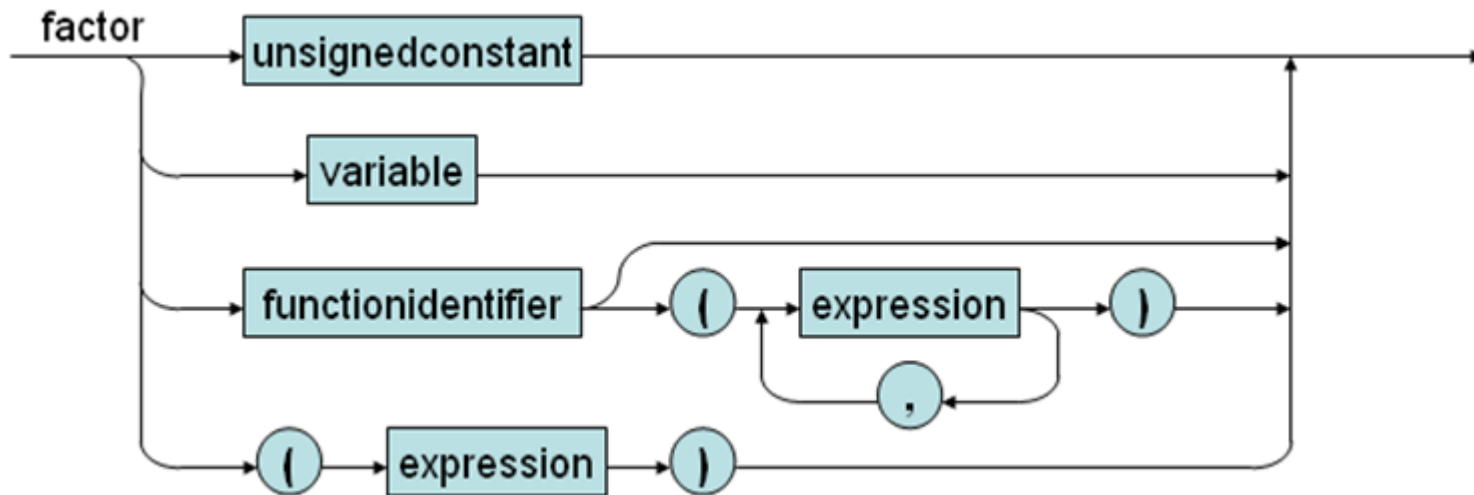


Combining

```
void compileTerm(void)
{
    compileFactor();
    while(lookAhead->tokenType == SB_TIMES || lookAhead->tokenType ==
        SB_SLASH)
    {
        switch (lookAhead->tokenType)
        {
            case SB_TIMES:
                eat(SB_TIMES);
                compileFactor();
                break;
            case SB_SLASH:
                eat(SB_SLASH);
                compileFactor();
                break;
        }
    }
}
```



Factor: Violation of LL(1)



$\text{FIRST}(\text{unsignedconstant}) = \{\text{TK_NUMBER}, \text{TK_IDENT}, \text{TK_CHAR}\}$

$\text{FIRST}(\text{variable}) = \{\text{TK_IDENT}\}$

$\text{FIRST}(\text{functioncall}) = \{\text{TK_IDENT}\}$

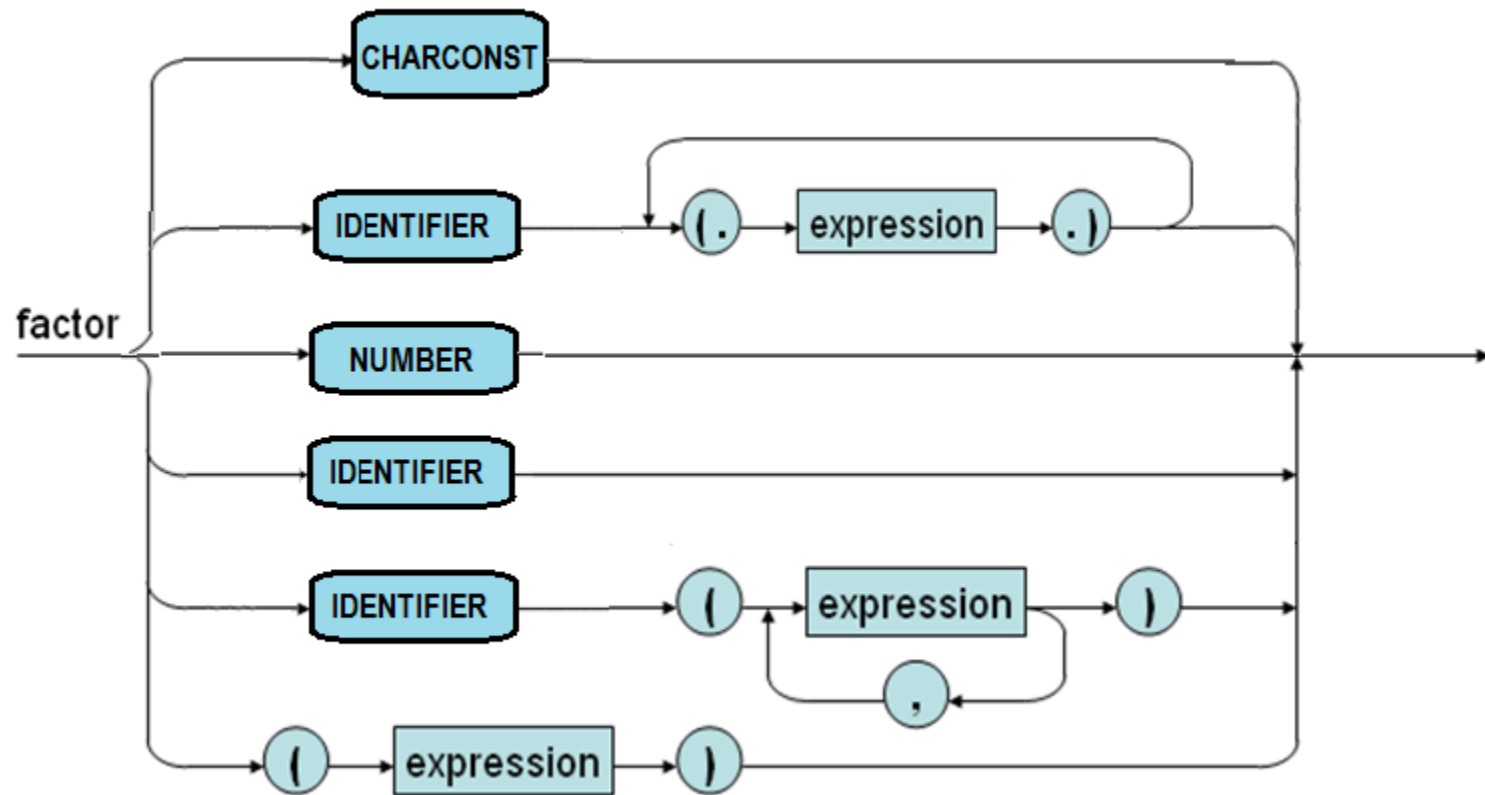
$\text{FIRST}(\text{unsignedconstant}) \cap \text{FIRST}(\text{functioncall}) = \{\text{TK_IDENT}\}$

$\text{FIRST}(\text{variable}) \cap \text{FIRST}(\text{functioncall}) = \{\text{TK_IDENT}\}$

$\text{FIRST}(\text{variable}) \cap \text{FIRST}(\text{unsignedconstant}) = \{\text{TK_IDENT}\}$

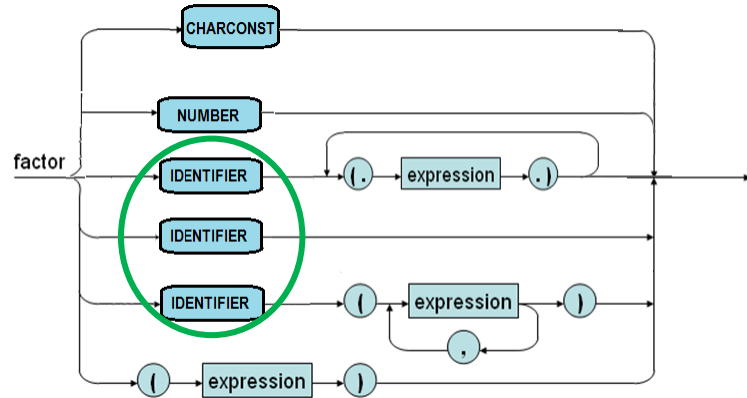
=> Violation of LL(1)

Analyzing factor



Phân tích factor

```
void compileFactor(void) {  
    switch (lookAhead->tokenType) {  
    case TK_NUMBER:  
        eat(TK_NUMBER);  
        break;  
    case TK_CHAR:  
        eat(TK_CHAR);  
        break;  
    case TK_IDENT:  
        eat(TK_IDENT);  
        switch (lookAhead->tokenType) {  
        case SB_LSEL:  
            compileIndexes();  
            break;  
        case SB_LPAR:  
            compileArguments();  
            break;  
        default: break;  
        }  
        break;  
    }
```



```
    case SB_LPAR:  
        eat(SB_LPAR);  
        compileExpression();  
        eat(SB_RPAR);  
        break;  
    default:  
        error(ERR_INVALIDFACTOR,  
            lookAhead->lineNo, lookAhead->colNo);  
    }  
}
```